Model & Platform Based Design of Embedded Systems

Raktim Bhattacharya
raktim@aero.tamu.edu
Aerospace Engineering Department
Texas A&M University
College Station, TX 77843-3141

April 7, 2006

Contents

1 Introduction **4**

2 Complexity in Real-Time Embedded Systems **4**
2.1 Environment . . . 5
2.2 Strictness of Deadlines . . . 5
2.3 Reliability . . . 5
2.4 Size and Degree of Coordination . . . 6
2.5 Fault Tolerance . . . 6

3 Platform Based Design **7**

4 The Role of Modeling & Formalism in Embedded System Design **8**
4.1 Mapping of Functionality to Architecture . . . 8
4.2 Consolidation of Hardware Platforms . . . 9
4.3 Reusability . . . 9
4.4 Virtual Prototyping . . . 10
4.5 Validation and Verification . . . 10
4.6 Software Design . . . 11
4.7 Hardware Design . . . 14

5 Rigorous Design Process for Embedded Systems **15**
5.1 Requirements Captured in Models . . . 16
5.2 Production Code Generated Automatically from Abstract Models . . . 17
5.3 Integration with Physical System . . . 17

6 Supporting Tools & Enabling Technology **17**
6.1 Software Modeling Tools . . . 17
6.2 Automatic Code Generation . . . 19
6.3 Requirement Documents Linking and Navigation . . . 19
6.4 Model Testing and Verification . . . 20
6.5 Hardware Modeling Tools . . . 21

7 Success Stories from Industries **21**

List of Figures

1 Paradigm Shift in Design and Implementation of Real-Time Embedded Systems 4
2 Design Space Exploration at Key Articulation Points in the Design Flow. 7
3 Separation of Concerns in Embedded Systems Design Process. 8
4 Mapping of Functionality to Architecture Using Code Generation Techniques. 9
5 Taxonomy of Hardware Modeling Tools [1]. 14
6 Paradigm Shift in Embedded System Design. (Image Source: PARADES) 16
7 Platform Based Design and the Design V. 16
8 Tools Supporting Platform Based Design Methodology. 18
9 Comparison of code generation tools. 20
10 Comparison of Requirements Management Tools 20

1 Introduction

In recent times, there has been a proliferation of embedded systems in our society. Embedded systems are found in diverse products ranging from cell phones to aircraft engines. The rapid growth in embedded systems is fuelled by the advancement in the micro-processor technology. Today, it is possible to build a complete computer system on a chip, including wireless Internet connectivity. These chips can easily be added to thousands of products, and soon will be. Computational hardware will continue to become cheaper, smaller and more powerful, and eventually they will be inexpensive enough to put in nearly every product.

The functionality of embedded systems is evolving from static dedicated systems to dynamic systems that adapt in real time to changes in the controlled system and its environment. The paradigm for system design and implementation is also shifting from a centralized, single processor framework, to a decentralized, distributed processor implementation framework. Distribution and decentralization of services and components is driven by the falling cost of hardware, increasing computational power, increasingly complex control algorithms and development of new, low cost micro-sensors and actuators. A distributed, modular hardware architecture offers the potential benefit of being highly reconfigurable, fault tolerant and inexpensive. Modularity can also accelerate the development time of products, since groups can work in parallel on individual system components. These benefits come with a price; the need for sophisticated, reliable software to manage the distributed collection of components and tasks, see Fig.1.

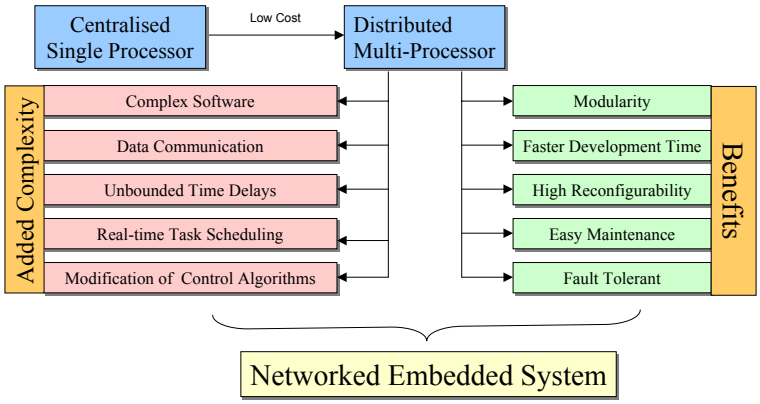


Figure 1: Paradigm Shift in Design and Implementation of Real-Time Embedded Systems

The growing complexity of realtime systems suggests a need for a rigorous framework for embedded system design that will meet performance, quality, safety, cost and time-to-market constraints. This is pushing the embedded systems community towards formulating a framework for a rigorous embedded system design process that caters to these needs.

2 Complexity in Real-Time Embedded Systems

Embedded systems are typically realtime systems that interact with the physical world. A realtime system is essentially a computational hardware that is executing several software elements in realtime. The utility of the realtime system is governed both by the functional correctness of the software it is running as well as the time it takes to complete the execution. We refer to the software elements in a realtime system as *computational tasks* and the constraint on the execution time as the *deadline*.

Complexity in a realtime system can be categorized in terms of the nature of the environment it interacts with, strictness of deadlines, reliability of the system, size and degree of coordination between various computational tasks in the system and fault tolerance.

2.1 Environment

The environment in which a realtime system operates, plays an important role in system design. Many environments are well defined and deterministic. These environments give rise to small, *static realtime systems* in which all deadlines can be guaranteed a priori. The approach taken in relatively small static realtime systems, however, do not scale to larger, more complicated and less controllable environments.

Owing to the benefits of modular design, many complex realtime systems are built by interconnecting sub-components. The behavior of the overall system is determined not only by the composite behavior of the sub-systems but also by the interconnection structure between these components. For example, in networked control systems [2, 3] the sub-components of the control systems communicate via a communication network. The environment, in which such control systems operate, is affected by network induced delays, packet loss, multiple packet transmission resulting in duplication of signal, etc. Clearly, determining the reliability of the realtime control system in such an environment is not trivial.

It is expected that future realtime embedded systems will be large, complex, distributed and dynamic. They will contain many types of timing constraints, precedence constraints and will need to operate in a fault-prone, highly non-deterministic environments. Formal analysis, validation and verification will be absolutely necessary to establish reliability of such *dynamic realtime systems*.

2.2 Strictness of Deadlines

Computational tasks occurring in a realtime system have timing constraints, or deadlines, which need to be satisfied for the realtime system to be functionally useful. A realtime system can be classified in three categories based on the nature of the deadlines they face. They could be *hard* realtime systems, if the consequences of not executing a task before its deadline is catastrophic. Flight control or control of nuclear plants are examples of hard realtime systems. Real-time systems are *firm*, if the consequences of missed deadlines are not severe. Online banking and airline reservation systems are firm realtime systems. A realtime system is categorized as a *soft* realtime system if the utility of the system degrades with time after the deadline expires. Real-time video streaming and telephone switching systems are examples of soft realtime systems.

2.3 Reliability

Real-time systems, like flight control avionics, operate under stringent reliability requirements. They are hard realtime systems and failure to meet the deadlines of the constituting scheduling tasks, may result in catastrophic consequences. An off-line scheduling analysis [4, 5] is usually conducted to ensure that the deadlines of all the tasks are met. Such an analysis is made subject to certain assumptions on the workload and failure conditions.

Scheduling of realtime tasks could be static where the order of execution of each task is pre-determined. It is not possible to change the task execution characteristics or even the execution order under such scheduling policy. Such systems are more deterministic in operation. It can be shown that static scheduling can achieve maximum hardware utilization of 69% [6], in the worst case.

This is in contrast to *dynamic scheduling* where it is guaranteed to utilize 100% of the hardware resources [6]. In dynamic scheduling, the task with the nearest deadline is given highest priority in the execution order.

Such systems can be configured to react to unexpected environmental behavior. The change in the execution characteristic of the task reacting to the unexpected environmental change can be accommodated using dynamic scheduling policy. Clearly, there are bounds on the amount of change that can be accommodated. Dynamic scheduling algorithms are more complicated to implement than static scheduling algorithms. The resulting realtime system is less deterministic.

Clearly, there is a tradeoff between the determinism, hardware utilization and reconfigurability.

2.4 Size and Degree of Coordination

Hardware realization of simple algorithms result in small realtime systems. The associated realtime tasks are independent of each other and the analysis of such systems for reliability is fairly simple. In most cases, the entire system code can be loaded into memory or if there are well-defined phases, each phase is loaded just prior to the beginning of the phase.

However, with the increasing role of information based systems (pg. 18 in [7]), the level of interaction and cooperation between the sub-tasks is on the rise. In recent times we are faced with situations in which pervasive computing, sensing and communication are common. Control and embedded system engineers are facing challenges of controlling large-scale systems and networks that result in large complex realtime systems and complicates the notion of reliability.

2.5 Fault Tolerance

Fault tolerance is defined as the system's ability to deliver the expected service even in the presence of faults. A realtime embedded system may fail to function properly either because of error in its hardware, software, or both, or because it fails to respond in time to meet the timing requirements demanded by the environment it interacts with.

There are different fault tolerant techniques in realtime embedded systems, namely:

- **N-Modular Redundancy (NMR)** - In this approach, N multiple identical processors concurrently execute the same task and the results produced by these processors are voted on by another processor. This is a general technique that can tolerate most of the hardware faults.
- **N-Version Programming (NVP)** - This approach is capable of tolerating both software and hardware faults. It is based on the principle of *design diversity*; i.e., a task is coded by different teams of programmers, in multiple versions.
- **Recovery Blocks** - This scheme uses multiple alternates to perform the same task. The various alternates are categorized as primary and secondary. The primary task executes first. Once it completes its execution, an acceptance test or a verification test is performed on its outcome. If the results, of the primary task fails the test, the secondary task executes after undoing the effects of the primary task (i.e., rolling back to the state at which the primary task was invoked) and so on. This continues until an acceptable result is obtained, all alternates are exhausted, or the deadline of the task is missed. This differs from NVP in executing the different versions of the task serially as opposed to parallel execution of versions in the NVP approach.
- **Imprecise Computations** - This approach avoids the timing faults during transient overloads by gracefully degrading the quality of the result via imprecise computations [8]. The imprecise computation model provides scheduling flexibility by trading off result quality to meet task deadlines. In this approach, a task is divided into a mandatory part and an optional part. The mandatory part must be completed before the task's deadline for an acceptable quality of result. The optional part, which can be skipped if necessary to conserve system resources, refines the result. A task is said to

have produced a precise result if it has executed both its mandatory and its optional parts before its deadline; otherwise it is said to have produced an imprecise result.

The overview of the functional characteristics of a realtime system, presented in this section is available in more detail in reference [9].

3 Platform Based Design

The PARADES group [10], a European consortium consisting of Cadence, Magneti-Marelli, ST Microelectronics, has developed a core set of technologies under the banner of *Platform Based Design*(PBD)[11], that defines a new paradigm in embedded system design. An essential component of the new system design paradigm is the *orthogonalization* of concerns, i.e., the separation of the various aspects of design to allow more effective exploration of alternate solutions.

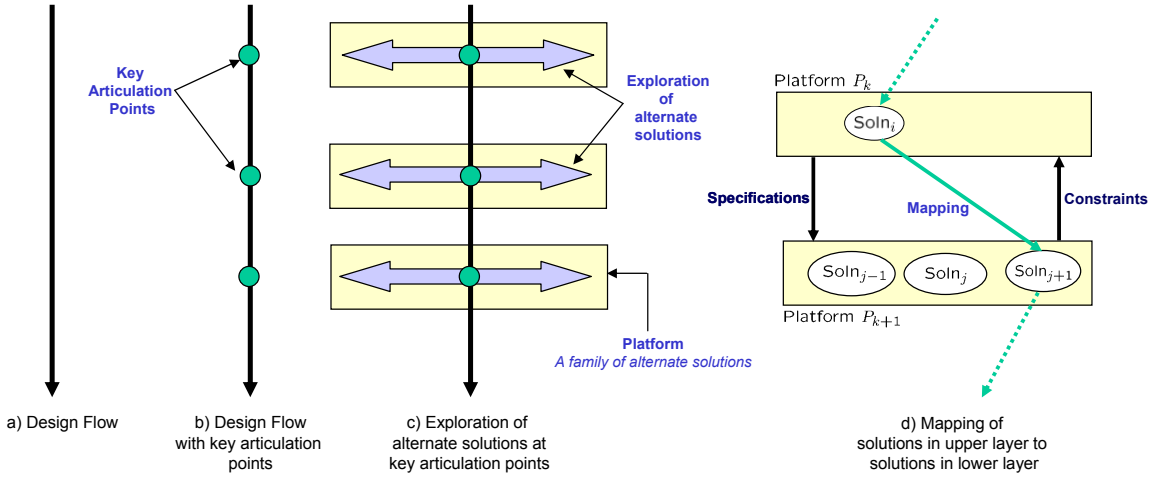


Figure 2: Design Space Exploration at Key Articulation Points in the Design Flow.

A *platform* is defined to be a family of alternate solutions at any key articulation point in the design process. Each platform represents a layer in the design process at which design space exploration is possible or desired. This is illustrated in Fig.2.

Solutions available at each layer are independent of solutions in other layers. For example, the solution Soln_i in platform P_k, in Fig.2(d), assumes no knowledge of the available solutions Soln_{j-1}, $\mathsf{Soln}j$, Soln_{j+1} in the lower platform P_{k+1}. This establishes a *separation of concern* between layers in the design flow.

In an abstract sense, one can imagine solutions in the upper layer to be definitions of *functionality* and the solutions in the lower layer to be definitions of *form*. The realization of a given functionality by means of one of the available forms, is the process of *mapping*. For the purpose of mapping functionality to form, it is only necessary to pass information to the lower layer that is relevant. This allows abstract characterization of functionality and enables design space exploration of form. For example, to realize the execution of a class of numerical algorithms in hardware, it is only necessary to define relevant information such as memory requirement, execution rate, data types (fixed or floating), etc. Details of the functional objective of the algorithm are not necessary for the purpose of mapping. Therefore, for this example all solutions of functionality can be *specified* abstractly to the lower layer in terms of parameters such as memory requirements, execution rate and data type. The solution space of hardware can also be expressed in terms of these same parameters which can define *constraints* on the functionality solution space. Fig.2(d) illustrates the process

of mapping solutions from one layer to another.

Fig.(3) illustrates the paradigm proposed by the PARADES group, which stresses the separation between:

- **Function** - defines what the system is supposed to do. The behavior of the system is expressed as the composition of functional processes communicating through media. The composition of functional processes is the *functional architecture*. The function layer, in general, only represents the system's behavior, and does not express any of the hardware implementation decisions. It is however restricted by the constraints imposed by the architecture layer below. Examples of such constraints are maximum communication bandwidth, available memory, quantization of data, clock speed, etc.
- **Architecture** - defines how it is realized. The architecture is made up of elements that will physically realize the functionality. These elements can be thought of as containers that support a variety of behaviors for given costs (e.g. time, power, area). The architectural elements support different types of functionality, but do not specify which ones will be used or when. The information necessary to realize the functionality of the system is provided by the function layer as specifications. The specifications contain information on the run-time characteristics of the algorithms that realize the functionality, such as sequence of execution, execution deadlines, priority, etc. Examples of architectural elements include hardware, microprocessors, digital signal processors, memories, buses, and reconfigurable logic.

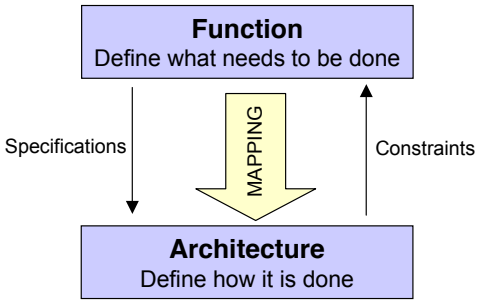


Figure 3: Separation of Concerns in Embedded Systems Design Process.

Each platform can be further decomposed into sub-layers if separation of concerns within a layer is needed. For example, in the function layer, the definition of the functional processes and their interconnection can be separated. This provides another degree of freedom to define the behavior of the system. In the architecture layer, one can define sub-layers that represent a hardware platform consisting of families of micro-processors and a software platform that defines the API(Application Programmer's Interface) to the hardware platform.

Once the platforms have been defined it is important to define the *mapping* of one platform to another. In the PARADES paradigm, the mapping of function to architecture is an essential step from conception to implementation. For more details on the PARADES framework, the reader is directed to reference [11].

4 The Role of Modeling & Formalism in Embedded System Design

4.1 Mapping of Functionality to Architecture

The mapping of one platform to another is facilitated if *formal descriptions* in the form of *models* are used to capture functions and architecture. Formal models are abstract representations of elements in a platform that are used during mapping of elements of one platform to another.

For example, functional processes in an elevator system can be modeled in Simulink, a graphical programming environment for developing control systems developed by Mathworks [12]. Once functionality has been defined as Simulink models, code generation tools such as TargetLink by dSPACE [13] or Real-Time Workshop by Mathworks, can be used to map functionality to specific hardware platforms. Fig.(4) illustrates the mapping of functionality to architecture in more detail for a generic system `mySystem`.

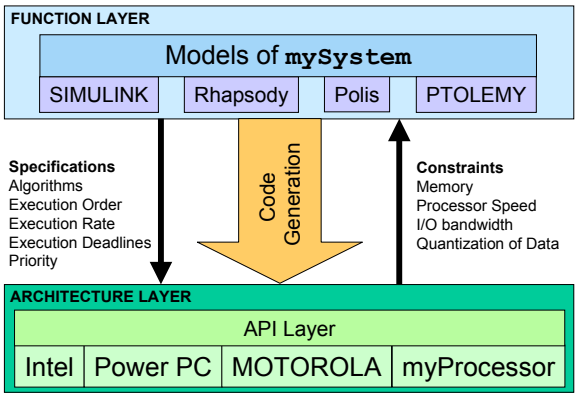


Figure 4: Mapping of Functionality to Architecture Using Code Generation Techniques.

We define *functional elements* of `mySystem` to be components such as motion controllers of an elevator, guidance algorithms in an aircraft, clutch control algorithms in an automobile, supervisory control in an air-conditioning system, etc. *Nonfunctional elements* of `mySystem` are defined as models of computation such as event driven or time-triggered, sampling rate, quantization elements such as floating or fixed point representation of data, sequence of computation, execution rate and deadlines, etc. Functional elements are defined by what the system is supposed to do. Nonfunctional elements are governed by how it is done or the constraints imposed by the available architecture. It is necessary that a formal description or the model of a given sub-system in `mySystem` must incorporate both these elements for a successful mapping of functionality to architecture.

The modeling of the functional elements of `mySystem` can be performed using tools such as Simulink or RHAPSODY [14]. The nonfunctional elements can be modeled using tools such as POLIS [15] or PTOLEMY [16]. A detailed discussion on tools and enabling technology for platform based design is presented later in this monograph.

4.2 Consolidation of Hardware Platforms

Often corporations grow by acquisitions of other companies. When companies dealing with embedded systems grow as a loose collections of many companies, a large number of hardware and software platforms emerge. Maintainability becomes an issue. Separating functionality from architecture allows consolidation of different hardware platforms. Different hardware platforms can be interfaced with a common high level API library providing a common gateway for the code generator to map functionality to architecture. This also reduces the sensitivity to technology obsolescence. New hardware platforms can be targeted by appropriately changing the API interface.

4.3 Reusability

Modeling also allows reuse. When functionality is described in terms of implementation it is interlaced with the implementation decisions the programmer makes during development. This makes it hard to identify commonality in software with other systems and impedes reuse. In the modeling framework, the models

of generic subsystems developed by various designers will result in a reusable library. The independence from hardware details will allow better understanding of the functional aspects of the subsystem modeled. Designers will be able to reuse their models when they design other systems or exchange it with other designers. Confidence in these models will increase over time as the code generated from these libraries is used in production, augmenting reusability.

4.4 Virtual Prototyping

Modeling environments like Simulink and RHAPSODY make the data flow semantics very clear. Subsystem interaction is clearly defined, which enables creativity. Designers are able to explore "what-if?" scenarios in system architecture very easily by rewiring the data flow, adding new functionality and creating a virtual prototype at their desktop. Most modeling environments allow simulation environments where these virtual prototypes can be tested via simulations. Designers are able to get better understanding of the system or the impact of a change very early in the design process. Without the modeling and simulation environment, designers would have to wait for the actual system to be built. Exploring "what-if?" scenarios at the physical prototype level has higher cost implications.

4.5 Validation and Verification

Reliability and performance of embedded systems is ensured by means of rigorous analysis. Analysis is necessary for the software elements, the hardware elements and the interaction of software with hardware in the embedded system. Details of software, hardware and hardware software co-verification is presented next.

1. **Software Verification** - The software analysis is partitioned into two parts, namely the *functional analysis*, which deals with the verification of the logic of the algorithm and *nonfunctional analysis*, which addresses the execution characteristics of the software.

   Functional analysis is focussed on the functional correctness of the algorithms that are executing in the system. Requirements specify the desired functional characteristics of the algorithms. Test scenarios that validate the algorithm against requirements can be easily created in the modeling environment and the functional correctness of the algorithm is established by conducting *simulations*.

   Algorithm verification done in the modeling framework also results in *higher confidence* in the test results. Often in traditional testing, algorithm functionality is tested on actual hardware. Such testing is usually prone to errors due to software-hardware interaction and is usually testing the algorithm in relation to the hardware. Such validation results are weak in terms of establishing reliability. Consider the design of a data communication protocol as an example. The protocol is expected to handle several errors that occur during transmission of data over the communication channel. In the modeling environment, it is possible to characterize each error, the time of occurrence and the sequence of occurrence. Often communication protocols are modeled as finite state machines, as errors are often defined by a sequence of events. The ability to precisely control the sequence of events in the modeling environment allows the designer to test the communication protocol completely and rigorously. This rigor is not possible if the algorithm is tested on a communication hardware, which is asynchronous in nature.

   Test plans that verify the algorithm against requirements can be defined by *input traces* or time trajectories of input signals. The corresponding output trajectories, defined to be *output traces*, are used to determine the functional correctness of the algorithm. For a correct design, these traces can be qualified to be *golden traces* and can be used to develop an *automatic regression testing* infrastructure. Regression testing in the modeling framework is defined to be the selective retesting of a model that has been modified to ensure that bugs have been fixed and that no other previously working models

have failed as a result of the reparations and that newly added features have not created problems with previous versions of the model. It is a quality control measure to ensure that the newly modified model still complies with its specified requirements and that unmodified parts of the model have not been affected by the maintenance activity.

In nonfunctional verification, the execution characteristics of the algorithm are analyzed. Sensitivity of the functional behavior to the changes in the execution characteristics can be analyzed. Some of the execution characteristics that can be easily captured in the modeling environment are models of computation which can be synchronous or asynchronous, data flow semantics that defines the input-output dependencies between subsystems and the verification of functional coverage.

2. **Hardware Verification** - Hardware modeling captures the behavior of the hardware in development in a simulation environment that is executable. This helps hardware developers in the physical design flow and also provides a software development platform for the purposes of co-verification. The model of the prototype hardware can execute on a desktop PC and is available to all members of the hardware development team. A virtual prototype of the hardware enables architectural exploration and a truly concurrent development process for hardware and software in which both are tightly linked back to architectural specification and system-level validation. The hardware model also takes the hardware prototype off the critical path and can save considerable project costs. It also leverages fast development cycle and increased instrumentation capability.

3. **Co-Verification** - The basic concept behind co-verification is to merge the respective debug environments used by hardware and software teams into a single framework. This provides designers with concurrent and early access to both the hardware and software components of the designs, thereby contributing to reducing the overall project cycle time. An efficient co-verification tool can help uncover a range of hardware-software interface problems, which include:

   - Initial startup and boot sequence errors (including RTOS boot)
   - Processor and peripheral initialization and configuration problems
   - Memory accessing and initialization problems
   - Memory map and register map discrepancies
   - Interrupt service routine errors

   By uniting the hardware and software simulation environments in a processor-based system, a co-verification tool can be conceptually viewed as an extension of traditional "functional simulation" in logic-only designs. The co-verification concept establishes value for multiple design teams including hardware engineers, embedded software engineers, and system designers [17].

By various accounts, design verification is the most serious bottleneck that engineers face in delivering reliable embedded systems. It is not uncommon for verification teams to spend as much as 50 to 70 percent of their time in verification and debug. The modeling framework for verification and validation offers a systematic verification process. Testing conducted in a simulation environment enables development of a regression testing capability, which can be partially automatized, if not fully. Automatic regression testing of the software, the hardware and the interaction of the two can result in considerable reduction in the verification time. For more details on verification of embedded systems, please refer to the white papers available at `http://www.verisity.com/resources/whitepaper/technical_paper.html`

4.6 Software Design

Model based design of algorithms has been a common practice among engineers. Engineers have used models of physical systems to design algorithms such as stabilizing controllers for aircraft, supervisory controllers for

HVAC systems, fault detection and isolation algorithms, filter design for signal processing applications, etc. Modeling environments like Dymola [18] are extensively used to model HVAC systems in diverse applications ranging from automotive to aerospace. Matlab & Simulink offer a powerful environment for modeling physical systems and design control or diagnostics algorithms. RHAPSODY is used to model complex software systems. In the platform based design framework, models can be used to explore alternate solutions in the model of computation used to design and implement relevant algorithms. Modeling can be used to analyze the functional coverage of the algorithm and also determine the limits of performance in non-ideal operating conditions.

Below we present details on models of computation, coverage analysis and limits of performance related to algorithm design.

1. **Models of Computation**(Ch. 7,[19]) - An algorithm can be designed using one or more of the several models of computation such as,

   - *Continuous Time*: Models are described by ordinary differential equations (ODE). The execution of such systems involve computation of a numerical solution to the ODEs at discrete set of time points. Interaction of such continuous time systems requires a careful control of the propagation of time by means of a strict execution order [20].
   - *Discrete Event*: In this model of computation components share a global notion of time and communicate through events that are placed on a continuous time line. Components react to events in a chronological manner. Such models exhibit *causal* execution characteristics. Systems such as communication networks, software timing properties and queuing systems are examples of such systems.
   - *Dataflow Models*[21, 22]: In dataflow models, connections represent data streams. Components map input data to output that gets propagated to components down stream. In such models the execution order is determined by the input data dependencies. Such models can be executed in a highly optimized manner or achieve parallelism. Example of such systems are signal processing algorithms or sampled control systems.
   - *Timed Multitasking*: In such models of computation the designer can explore priority based scheduling policies and study their effects on the realtime software. Algorithms are executed according to priorities set by the designer. An algorithm can get *preempted* by a higher priority algorithm. The preempted algorithm freezes in time while the higher priority algorithm executes. On completion of the execution of the higher priority algorithm, the preempted algorithm continues to execute. The assigned priorities can be *static* if they don't change over time and *dynamic* otherwise. When executions are not preempted the model of computation is referred to as *non-preemptive*. Modeling environment such as PTOLEMY II allows modeling of both preemptive and nonpreemptive scheduling as well as static and dynamic priority assignment.
   - *Synchronous/Reactive*: In synchronous/reactive (SR) models of computation [23], the connections represent signals whose values are aligned with global clock ticks. This is in contrast to data flow models where there is no time constraints. Examples of languages that use the SR model of computation are Giotto [24], Esterel [25] and Signal [26]. Due to the tight synchronization, safety-critical realtime applications often use this model of computation.
   - *Finite State Machines*: Finite state machines (FSMs) are intuitive models of sequential logic and the discrete evolution of physical processes. An FSM model has states and transitions among states. It reacts to input by taking a transition from its current state. Output may be produced by actions associated with transitions. FSM models are amenable to in-depth formal analysis and verification. Data path controllers in microprocessors and communication protocols are examples of applications of FSM models.

The modeling environment provides the ability to explore various models of computation in the design of an algorithm. The impact of a choice of a model of computation can be simulated in the modeling environment. Reliability is assessed at the modeling level resulting in early validation.

2. **Coverage Analysis** - When embedded systems are constructed using multiple subsystems, the combined complexity of the multiple sub-systems can be huge. There are many seemingly independent activities that need to be closely correlated. It is necessary to validate the integrated system, even if the subsystems are assumed to have been verified. Testing of the integrated system based on the requirements may leave certain areas not validated. Coverage analysis is conducted for identifying areas that were never exercised.

   People tend to think of code coverage when they speak about coverage analysis. Code coverage is a good "first pass" indication for areas that were never covered. It cannot determine if all the "interesting" scenarios were exhausted. Classic code coverage analysis can only be judged as a necessary criterion, not as a sufficient one. It cannot determine whether all specified features are properly implemented; it can only determine that all implemented code has been sufficiently verified. What is required is a specification-driven functional coverage.

   Functional coverage allows the designer to define exactly what functionality of the device should be monitored and reported. Therefore, for functional coverage analysis it is necessary to execute the test plan. The modeling and simulation environment allows virtual prototyping where functional coverage analysis can be easily conducted. Without the modeling environment, such analysis cannot be done until the real prototype is available. The modeling environment also provides visibility into the execution characteristics that are to be monitored.

   The following are the common coverage analysis that are typically conducted [12]:

   - *Cyclomatic complexity*: It is a measure of the structural complexity of a model. It approximates the McCabe [27, 28] complexity measure for code generated from the model. In general, the McCabe complexity measure is slightly higher because of error checks that the model coverage analysis does not consider. The Model Coverage Tool uses the following formula to compute the cyclomatic complexity of an object (block, chart, state, etc.):

     $$c = \sum_{1}^{N} (o_n - 1)$$

     where N is the number of decision points that the object represents and on is the number of outcomes for the nth decision point.

   - *Decision coverage*: examines items that represent decision points in a model. For each item, decision coverage determines the percentage of the total number of simulation paths through the item that the simulation actually traversed.

   - *Condition coverage*: Examines blocks that output the logical combination of their inputs, e.g., the Logic block, and state transitions. A test case achieves full coverage if it causes each input to each instance of a logic block in the model and each condition on a transition to be true at least once during the simulation and false at least once during the simulation. Condition coverage analysis reports for each block in the model whether the test case fully covered the block.

   - *Modified condition/decision coverage (MC/DC)*: Examines blocks that output the logical combination of their inputs (e.g., the Logic block) and state transitions to determine the extent to which the test case tests the independence of logical block inputs and transition conditions. A test case achieves full coverage for a block if, for every input, there is a pair of simulation times when changing that input alone causes a change in the block's output. A test case achieves full

coverage for a transition if, for each condition on the transition, there is at least one time when a change in the condition triggers the transition.

- *Lookup table (LUT) coverage*: Examines blocks, such as the 1D Look-Up block, that output the result of looking up one or more inputs in a table of inputs and outputs, interpolating between or extrapolating from table entries as necessary. Lookup table coverage records the frequency that table lookups use each interpolation interval. A test case achieves full coverage if it executes each interpolation and extrapolation interval at least once. For each LUT block in the model, the coverage report displays a colored map of the lookup table indicating where each interpolation was performed.

3. **Limits of Performance** - Algorithms are often designed with assumptions that may not be valid in the field. For example, control system designers assume instantaneous transfer of signals from one component to another. In reality, there are delays due to congestion in the communication hardware. In a multi-tasking environment, depending on how asynchronous the system is, it is possible that the computation of an algorithm maybe delayed. There may be transients in the computational load and certain computations may not be done within the specified deadline. Such deviation from design assumptions cause degradation in the performance of the algorithm. It is possible to capture these scenarios in the modeling environment and assess the limits of performance of the embedded system in simulation.

To summarize, model based software design offers a systematic design of critical applications with quantifiable metrics on performance, reliability and the operation envelope.

4.7 Hardware Design

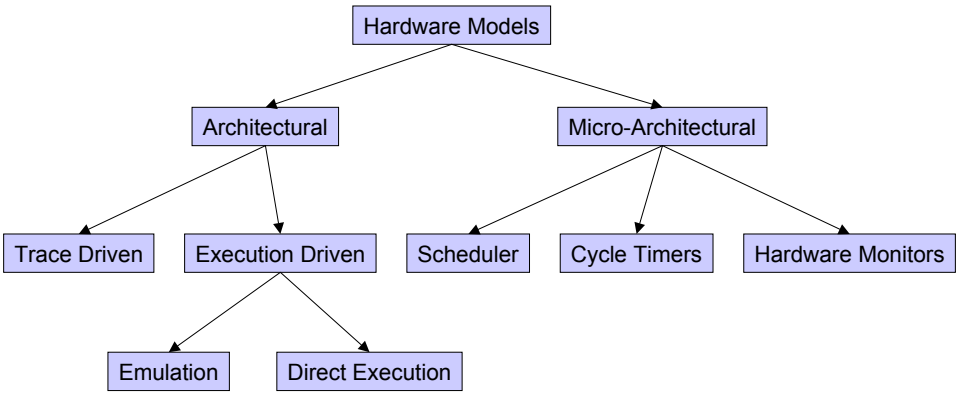


Figure 5: Taxonomy of Hardware Modeling Tools [1].

Fig.(5) illustrates the taxonomy of hardware modeling approaches [29]. They are categorized into two broad classes, *architectural* and *micro-architectural*.

1. **Architectural Modeling** - Architectural modeling relies on *trace-driven* or *execution-driven* modeling framework, both in a simulation environment.

   - *Trace Based Modeling*: In trace based modeling, the simulator reads a "trace" of instructions or a stream of prerecorded instructions to drive a hardware timing model. This method uses a variety of hardware and software based techniques - such as hardware monitoring, binary instrumentation

or trace synthesis- to collect instruction traces. It is the easiest to implement as it requires no functional component.

- *Execution Driven Modeling*: In execution driven simulations, the simulator "runs" the program, generating "traces" on the fly. It is more difficult to implement as it requires instruction functions and input/output handling. This approach has many advantages. It is able to provide access to all data produced and consumed during program execution. These values are crucial to the study of optimizations such as value prediction, compressed memory systems and dynamic power analysis. For example, in dynamic power analysis, the simulation must monitor the data values sent to all micro-architectural components such as arithmetic logic units and caches to gauge dynamic power requirements. The hamming distance, of consecutive data inputs defines the degree to which input bits change, which in turn causes transistors switching that consumes dynamic power. Execution driven simulation also permits greater accuracy in the modeling of speculation, an aggressive performance optimization that runs instructions before they are known to be required by predicting vital program values such as branch directions or load addresses. Speculative execution proceeds at a higher throughput until simulation finds an incorrect prediction. At this point, the processor pipeline is flushed and restarted with correct program values. Trace driven models cannot model misspeculated code execution because the trace only captures correct program execution. Execution driven approaches, on the other hand, can faithfully reproduce the speculative computation and correctly model its impact on program performance.

  Execution driven simulation has two potential drawbacks: increased model complexity and inherent difficulties in reproducing experiments. Model complexity increases because execution driven models require instruction and I/O (input/output) emulators to produce program computation, while trace based models do not. Execution driven simulators interface directly to input devices through I/O emulation, reproducing experiments that depend on real world external events may be difficult.

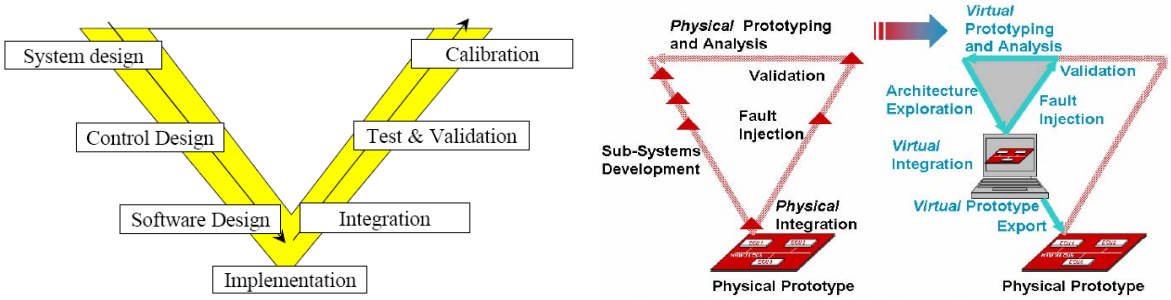
2. **Micro-architectural Modeling** - As shown in Fig.(5), micro-architectural modeling involves:

   - *Constraint-Based Instruction Schedulers*: Instructions are scheduled into execution graph based on available resources. The instructions are executed one at a time and in order. Such models are simple to modify but usually less detailed and limiting.
   - *Cycle-Time Simulators*: The model tracks detailed micro-architectural state for each cycle. Multiple instructions may be executed at a given time. Such models represent faithful representation of the hardware. The simulator state is equivalent to the state of the micro-architecture.
   - *Hardware Performance Monitors*: Such models capture and count hardware events probed by the system software. This provides a great framework for design postmortem and software performance analysis. It is however very limiting for next generation hardware design.

To summarize, the role of modeling in the hardware design is to conduct analysis related to the hardware such as dynamic power consumptions, design and analysis of speculative execution of code - an aggressive performance optimization technique, resource usage, design of experiments for post-mortem, hardware-software co-verification and software analysis. Such analysis results a optimized and reliable design which has direct cost implications. It also enables growth towards next generation hardware architectures.

5 Rigorous Design Process for Embedded Systems

Traditional V cycle, as shown in Fig.6(a) for the automotive industry, is purely top-down. It is good for new functionality but does not take into account available solutions. The bottom-up approach is good for strong component reuse, but it becomes difficult to guarantee the functional requirements of the integrated system.



(a) Traditional Design V for Automotive Industry

(b) Shift from Physical Prototyping and Integration to Virtual Prototyping and Integration

Figure 6: Paradigm Shift in Embedded System Design. (Image Source: PARADES)

In the platform based design framework, embedded system design is shifting from physical prototyping and integration to virtual prototyping and integration, as illustrated in Fig.6(b). Virtual prototyping is enabled by the modeling framework discussed earlier in the monograph.

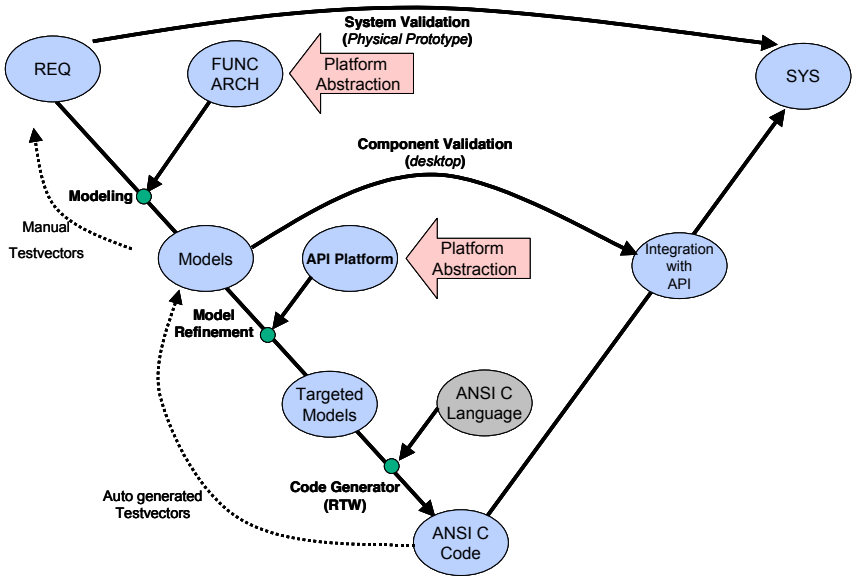


Figure 7: Platform Based Design and the Design V.

The platform based design is represented as a design V in Fig.7. The design V can be partitioned into three activities as described below.

5.1 Requirements Captured in Models

The main steps involved in this process are to map requirements to models. Since a given functional requirement can be implemented in many ways, it is necessary to define the functional architecture. Thus there is a *degree of freedom* in selecting the desired functional architecture. A designer may choose to implement several architectures during the modeling of the requirements. The down selection is governed by analysis similar to Failure Mode and Effect Analysis (FMEA) or other reliability analysis. We qualify functional

architecture as a platform since alternate solutions are possible at this point of design.

Once the models have been developed, the functional requirements are linked to the models. Simulink verification and validation toolbox allows the designer to link requirements specified in Microsoft Word or DOORs [30] to models.

The models developed are then validated against the requirements. Input traces are created that define the test environment for a given requirement. The models executed in simulation map the input traces to output traces. The output traces are visually inspected to verify correct implementation of the requirements. Once the models satisfy the requirements, the input and output traces are qualified as "golden traces", which are used in regression testing downstream in the design flow. Model validation process is represented in Fig.7 by the dotted arrow going from "Models" to "REQ". The text "Manual Test Vectors" on the arrow, represents the manually generated input traces that define the test environment.

The models now represent the requirements without any ambiguity and are strongly coupled with the functional architecture chosen. This refines the models one step closer to the real implementation. The model of the system is the virtual prototype.

5.2 Production Code Generated Automatically from Abstract Models

Once requirements have been captured as abstract models, code generation technology is used to translate functionality, defined in models, to production code. Today, the code generation technology is still under development. The mapping of the models to code is required to be validated. Modelling environments like Matlab allow verification of the generated code in simulation. The input and output traces that qualified the models can be used to verify the generated code as well.

The mapping of models to code allows various flexibilities such as data type abstraction, polymorphic operators, float-to-fixed point translations, target data-type refinement, etc. It is possible to synthesize entire applications by mapping sub-systems to realtime tasks.

5.3 Integration with Physical System

By formally defining the interface of the models with the software API, code generation technology can be used to integrate the automatically generated code directly to the API. This reduces errors due to interface. System wide testing is then conducted to exercise hardware-software interactions. With the proper hardware-in-the-loop testing facility and hardware modeling capability the number of errors during this phase can be significantly decreased.

6 Supporting Tools & Enabling Technology

In this section we present the tools and enabling technology that support platform based design framework. The discussion here is not expected to be exhaustive in any way. It is only an introduction.

Fig.8 summarizes the tools landscape that support the platform based design framework. Tools for functional design are more matured than those supporting hardware selection and analysis. Public domain hardware analysis tools are still at the research level.

6.1 Software Modeling Tools

- **MathWorks** [12]: Mathworks has developed a set of tools for design & analysis of functional elements. They are described as follows:

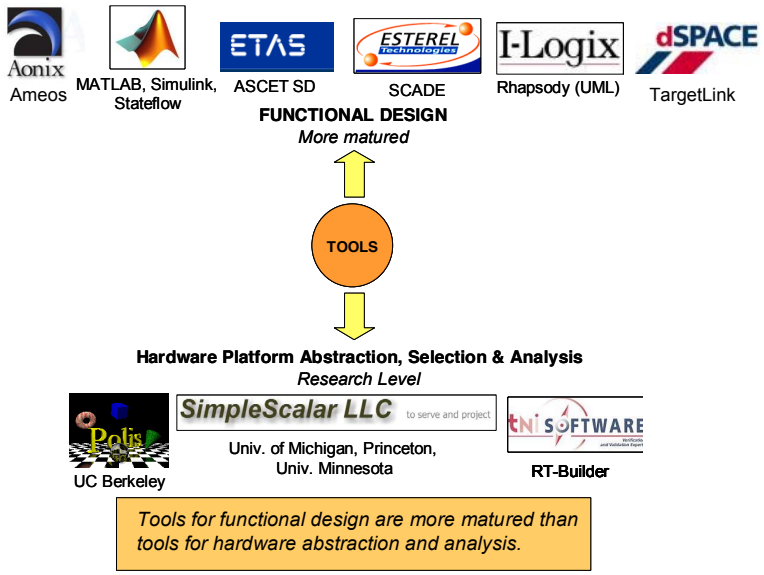


Figure 8: Tools Supporting Platform Based Design Methodology.

- *Matlab* is a high-level language and interactive environment that enables designers to perform computationally intensive tasks faster than with traditional programming languages such as C, C++, and Fortran.

- *Simulink* is a platform for multi-domain simulation and Model-Based Design for dynamic systems. It provides an interactive graphical environment and a customizable set of block libraries, and can be extended for specialized applications.

- *Stateflow* is an interactive design and simulation tool for event-driven systems. Stateflow provides the language elements required to describe complex logic in a natural, readable, and understandable form. It is tightly integrated with Matlab and Simulink, providing an efficient environment for designing embedded systems that contain control, supervisory, and mode logic.

- **Rhapsody** by I-Logix [14]: It is the industry's leading Model-Driven Development environment based on UML 2.0 [31] and SysML [32] for systems, software, and test, and has the unique ability to extend its modeling environment to allow both functional and object oriented design methodologies in one environment.

- **Ameos** by Aonix [33]: Ameos is also a UML based modeling framework. It implements the current UML standard and can be used to describe business processes, to design architectures for software systems and model dynamic aspects in state machines with hard timing constraints. The Model management of the UML is an integrated part of Ameos and allows distributed working, private workspaces and the configuration of new versions. The Ameos Multi-User Repository ensures an appropriate scaling even in large projects.

- **ASCET** by ETAS [34]: The ASCET product family consists of the high quality, rapid development tools ASCET-MD, ASCET-RP, and ASCET-SE, all of which ensure the reusability of existing software components. The well-designed graphical development environment of ASCET-MD enables target-independent functional specifications. ASCET-SE offers automated generation of control software from block diagrams and state machines. ASCET-RP facilitates target identical rapid prototyping,

and offers the ability to perform tests at every stage of the development cycle. The result is a reusable specification from which safe target-specific executable production code can be generated by a mouse click.

- **SCADE** by Esterel [25]: SCADE stands for **S**afety-**C**ritical **A**pplication **D**evelopment **E**nvironment.

  SCADE is the acknowledged market leader for the development of critical embedded software for the Civilian Avionics industry and the emerging standard in the Automotive industry, enabling the creation of unambiguous specifications and solving the "late change" problem by automating the specification to implementation flow.

  - Covers the full development cycle of critical embedded software from specifications to the generation of correct by-construction production code in C and Ada.
  - Supports both data flow and control logic type of applications.
  - The only commercial automatic code generation tool qualified to the strictest level of the Civilian Avionics Standard DO-178B, Level A.

Many of the tools mentioned offer gateways to Matlab/Simulink suite, resulting in a heterogenous model development environment.

6.2 Automatic Code Generation

Automatic code generation tools are critical in the platform based design framework. It is used to map functionality defined in abstract models to architecture. A list of tools and providers is listed below.

Mathworks provides three code generation tools. *Real-Time Workshop* generates and executes stand-alone C/C++ code for developing and testing algorithms modeled in Simulink. The generated code in C++ is not object-oriented and is very similar to C code. The resulting code can be used for many realtime and non-realtime applications, including simulation acceleration, rapid prototyping, and hardware-in-the-loop testing. It is possible to interactively tune and monitor the generated code using Simulink blocks and built-in analysis capabilities, or run and interact with the code outside the Matlab and Simulink environment. *Real-Time Workshop Embedded Coder* generates C/C++ code from Simulink and Stateflow models that has the clarity and efficiency of professional handwritten code. The generated code is exceptionally compact and fast, an essential requirement for embedded systems, on-target rapid prototyping boards, microprocessors used in mass production, and realtime simulators. Real-Time Workshop Embedded Coder can be used to specify, deploy, and verify production-quality software. *Stateflow Coder* generates portable integer, floating-point, or fixed-point C code for Stateflow charts. It supports all Stateflow objects and semantics, helping you develop and test algorithms that can be deployed as stand-alone applications or inserted as subroutines into your existing code. The code generation is limited to ANSI C/C++ language only.

TargetLink, the production code generator from dSPACE [13], generates production code straight from Simulink and Stateflow models. TargetLink's modeling environment is essentially MathWorks' Simulink/Stateflow package along with the limited number of additional Simulink blocks specifically developed by dSPACE. Currently, TargetLink can only generate ANSI C code.

Modeling tools such as Rhapsody, SCADE, ASCET and Ameos packages all have code generation capability. A comparison of the code generation capability is summarized in Fig.9.

6.3 Requirement Documents Linking and Navigation

Matlab/Simulink/Stateflow, Rhapsody and Ameos packages offer a direct link between the requirement documents, models and further down to automatically generated code.

Code Generator	Language
Mathworks	C,C++
dSPACE TargetLink	C
Rhapsody	C, C++, Java, Ada
Ameos	C, C++, Java, Ada
ASCET	C
SCADE	C, Ada

Figure 9: Comparison of code generation tools.

Rhapsody offers a *bidirectional* link between the model and requirement documents in Telelogic DOORS [30]. It also provides *unidirectional* interface between model and requirements document. The requirements document format supported includes Microsoft Word/Excel, Adobe Acrobat portable document format (PDF), text files, IBM Requisite Pro and UGS Slate.

Mathworks add-on software package, Simulink Verification & Validation Toolbox, also provides a bidirectional link with the requirements document in Telelogic DOORS and Microsoft Word/Excel format. Adobe PDF format, Text, and HTML files are only supported with a unidirectional interface

Ameos software package has unidirectional interface with Telelogic DOORS. The tools that support requirements management are summarized in Fig.10.

Modeling Environment	Unidirectional Link	Bidirectional Link
Mathworks	Adobe PDF, Text Files, HTML	DOORS, Microsoft Word/Excel
Rhapsody	Microsoft Word/Excel, Adobe PDF, Text Files, IBM Requisite Pro, UGS Slate	DOORS
Ameos	DOORS	None

Figure 10: Comparison of Requirements Management Tools

6.4 Model Testing and Verification

Mathworks Simulink Verification & Validation (V&V) toolbox is used to develop requirements based test designs and test cases as well as to measure test coverage. The V&V toolbox provides model coverage reports that indicate untested elements in the system such as unexecuted subsystems, unselected switch positions or untraversed conditional transition paths. Coverage results are reported directly in HTML documents. Test vector generation capability is currently not provided by Mathworks. Tools such as *Reactis* by Reactive Systems [35] and *Safety Test Builder* by TNI-Valiosys [36] can be used to generate test cases from Simulink/Stateflow models for the purpose of validating both the model and the automatically generated code.

Rhapsody offers model driven test generation and requirements based testing tools. These are provided by the software packages, *Automatic Test Generator* (ATG) and *Test Conductor*. Test Conductor enables testing and model verification based on the user specified input vectors. ATG on the other hand, generates test cases with a high model coverage, including states, transitions, operations, and generation of events. It also includes all relevant input combinations for modified condition/decision coverage (MC/DC) analysis.

Ameos software package offers Test Harness with graphical interface for data driven Test Case Generation. MC/DC analysis is also possible with this package.

6.5 Hardware Modeling Tools

Hardware modeling tools are still in the research level. We present a brief description of the state-of-the-art in hardware modeling.

POLIS [15] and its successor *METROPOLIS* [37] are tools currently developed at UC Berkeley that offers the technology for hardware software codesign. Metropolis consists of an infrastructure, a tool set, and design methodologies for various application domains. The infrastructure provides a mechanism such that heterogeneous components of a system can be represented uniformly and tools for formal methods can be applied naturally.

The *SimpleScalar* [38] tool set is a system software infrastructure used to build modeling applications for program performance analysis, detailed microarchitectural modeling, and hardware-software co-verification. Using the SimpleScalar tools, users can build modeling applications that simulate real programs running on a range of modern processors and systems. The tool set includes sample simulators ranging from a fast functional simulator to a detailed, dynamically scheduled processor model that supports non-blocking caches, speculative execution, and state-of-the-art branch prediction. The SimpleScalar tools are used widely for research and instruction, for example, in 2000 more than one third of all papers published in top computer architecture conferences used the SimpleScalar tools to evaluate their designs. In addition to simulators, the SimpleScalar tool set includes performance visualization tools, statistical analysis resources, and debug and verification infrastructure.

Other research groups involved in developing tools for hardware modeling, verification and analysis include the formal verification group at Stanford who have developed *Cooperating Validity Checker* and *Murphy* [39], *MOCHA* [40] by University of Pennsylvania - is a growing interactive software environment for system specification and verification, the SPIN Model Checker [41].

Most hardware based companies like STMicroelectronics and Xilinx have in house hardware modeling tools that are used for product verification and performance optimization.

7 Success Stories from Industries

Model and platform based design has achieved considerable success in various embedded systems applications including aerospace, automotive, communication, biomedical, chemical, semiconductor, etc.

In the aerospace industry, Airbus has significantly decreased number of coding errors due to the extensive use of automatic code generation from the model using SCADE [42]. For the Airbus A340 project, the ratio of automatically generated code reached up to 70%. Honeywell cut design time by 60% by using Matlab and Simulink from MathWorks. Honeywell also reported automatically generating 1.6 million lines of C code in the years 2003 and 2004, related to aerospace applications, using MathWorks' Real-Time Workshop Embedded Coder. Generated code contained only one error [43]. Lockheed Martin Aeronautics company made a

multi-million dollar purchase of I-Logix's Rhapsody software suite products in 2002, with the objective of using it in the second phase of their Joint Strike Fighter F-35 (JSF) project [14]. In BAE Systems, Matlab and Simulink greatly reduced development cycle time and cut system and software design and testing costs by 50%. In 2003, Air Systems Business Group of BAE Systems also ordered over $1 million in I-Logix's Rhapsody and Statemate products, to be used across numerous aerospace and military programs. NASA used Stateflow and Stateflow Coder to generate fault-protection code for deep space 1. European companies like Eurocopter and First Flight have also experienced similar benefits.

Automotive industry is an area where different aspects of rigorous design of embedded systems are already widely accepted and used. Model based design, automatic code generation, verification and validation, for example, are currently being used by numerous car manufacturers, including BMW, Porsche, Jaguar, Volvo, Ford, Toyota, Toyota, Honda, Audi, GM, Nissan, Renault, Peugeot Citroen. Application areas within this industry are numerous and include [13, 12, 44, 45]:

- Powertrain applications (e.g., air/fuel ratio, exhaust gas, transmission, and electronic throttle control)
- Vehicle dynamics (e.g., braking system, Electronic Stability Program - ESP, and steering system)
- Safety-critical and safety-related applications (e.g., air bags deployment, seat belt pretensioners, tire pressure monitoring, and rollover detection)
- Body applications (e.g., headlight adjustment and climate control)
- Driver assistance systems (e.g., cruise control)

Modeling environment for most of the above applications is Mathworks Simulink/Stateflow suite, and automatic code generation tools mainly used include dSpace's TargetLink and MathWorks Real-Time Workshop (RTW) and RTW Embedded coder. Of the two, dSpace is a company very much focused and specialized in the automotive area, and as much as 75% of its sales originate from the automotive sector. Using dSpace's TargetLink, it is possible to generate a generic (ANSI C) code, as well as a processor specific C code, targeting Hitachi SH-2/SH-2E, Infineon C16x and Tricore, Motorola 683xx, Mitsubishi M32R, Motorola MPC5xxv processors. The tool has been thoroughly tested and proved its code generation capabilities in various applications within the automotive sector. In addition to MathWorks' and dSpace's software packages, I-Logix's Rhapsody in MicroC is also being used by Nissan and Saab companies for designing body electronics systems applications [14].

Platform based design (PBD) methodology, as a way of doing a rigorous design of embedded systems, has also been gaining a wider and wider acceptance within the automotive sector. Ford, GM, Daimler Chrysler, BMW, and Magneti Marelli (supplier for the world's major car manufacturers) have all implemented PBD methodology for different applications within automotive field. Magneti Marelli Powertrain, for example, defined a highly configurable and efficient software platform covering a set of applications. Availability of such a platform enabled the development of control models and application software components independent of Electronic Control Unit (ECU) hardware version. In this way, it was possible to use the same software platform library across different car engines. Implementation of platform and model based development led to a significant increase in the application software productivity in terms of the software lines of code (SLOC) produced per hour (as high as 4 times faster production in comparison to a traditional hand-coding cycle) [46].

References

[1] Todd Austin. Hardware modeling infrastructure: The simplescalar experience. `http://research.microsoft.com/si/PPT/HardwareModelingInfrastructure.pdf`.

[2] L. G. Bushnell. Networks and control. *IEEE Control Systems Magazine*, 21(1):22–23, Feb 2001.

[3] W. Zhang, M. S. Branicky, and S. M. Phillips. Stability of networked control systems. *IEEE Control Systems Magazine*, 21(1):84–99, Feb 2001.

[4] D. Seto, J. P. Lehoczky, L. Sha, and K. G. Shin. On Task Schedulability in Real-Time Control System. *IEEE Real-Time Systems Symposium*, pages 13–21, Dec. 1996.

[5] K. Ramamritham and J. A. Stankovic. Scheduling Algorithms and Operating Systems Support for Real-Time Systems. *Proceedings of IEEE*, 82(1):55–67, January 1994.

[6] C. L. Liu and J. W. Layland. Scheduling Algorithms for Multiprogramming in a Hard Real-Time Environment. *Journal of Association for Computing Machinery*, 20(1):46–61, Jan 1973.

[7] R. Murray. Control in an Information Rich World. *Report of the Panel on Future Directions in Control, Dynamics, and Systems*, June 2002.

[8] J.W. S. Liu, K. J. Lin, W. K. Shih, A. C. Yu, J. Y. Chung, and W. Zhao. Algorithms for Scheduling Imprecise Computations. *IEEE Computer*, 24(5):58–68, 1991.

[9] C. S. R. Murthy and G. Manimaran. *Resource Management in Real-Time Systems and Networks*. The MIT Press, Cambridge, Massachusetts, 2001.

[10] A. Sangiovanni-Vincentelli. 1996 to 2004 Research Activities: PARADES(Project for Advanced Research of Architecture and Design of Electronic Systems). *Technical Report,PARADES*, 2004.

[11] A. Ferrari and A. Sangiovanni-Vincentelli. System Design: Traditional Concepts and New Paradigms. *Proceedings of 1999 International Conference on Computer Design: VLSI in Computer and Processors, ICCD'99, Austin, TX, USA*, pages 10–13, Oct 1999.

[12] Matlab and Simulink. `http://www.mathworks.com`.

[13] dSpace. `http://www.dspaceinc.com/ww/en/inc/home.htm`.

[14] Rhapsody. `http://www.ilogix.com/`.

[15] POLIS. `http://www-cad.eecs.berkeley.edu/Respep/Research/hsc/abstract.html`.

[16] Ptolemy. `http://ptolemy.eecs.berkeley.edu/`.

[17] Co-Verification. `http://www.fpgajournal.com/articles_2005/20050222_mentor.htm`.

[18] Dymola. `http://www.dynasim.com/`.

[19] X. Liu, J. Liu, J Eker, and E. A. Lee. *Software Enabled Control - Information Technology for Dynamical Systems*. Wiley Interscience, 2003.

[20] J. Liu and E. A. Lee. Component-Based Hierarchical Modeling of Systems with Continuous and Discrete Dynamics. *IEEE Symposium on Computer-Aided Control System Design (CACSD'00), Anchorage Alaska*, Sep 2000.

[21] E. A. Lee and D. G. Messerschmitt. Synchronous Data Flow. *Proceedings of the IEEE*, Sep 1987.

[22] E. A. Lee and T. M. Parks. Dataflow Process Networks. *Proceedings of the IEEE*, 83(5):773–801, May 1995.

[23] S. A. Edwards. *The Specification and Execution of Heterogeneous Synchronous Reactive Systems*. PhD thesis, U. C. Berkeley, May 1997. Available at UCB/ERL M97 `http://ptolemy.eecs.berkeley.edu/papers/97/sewardsThesis/`.

[24] T. A. Henzinger, B. A. Horowitz, and C. M. Kirsh. Giotto: A Time Triggered Language for Embedded Programming. *Proceedings of the First International Workshop on Embedded Software*, 2211:175–194, 2001.

[25] G. Berry and G. Gonthier. The Esterel Synchronous Programming Language: Design, Semantics, Implementation. *Science of Computer Programming*, 12(2):87–152, 1992.

[26] A. Benveniste and P. Le Guernic. Hybrid Dynamical Systems Theory and the SIGNAL language. *IEEE Transactions on Automatic Control*, 35(5):525–546, 1990.

[27] Thomas J. McCabe and Charles W. Butler. Design Complexity Measurement and Testing. *Communications of the ACM*, 32(12):1415–1425, December 1989.

[28] Thomas J. McCabe and Arthur H. Watson. Software Complexity. *Crosstalk, Journal of Defense Software Engineering*, 7(12):5–9, December 1994.

[29] T. Austion, E. Larson, and Dan Ernst. SimpleScalar: An Infrastructure for Computer System Modeling. *IEEE Computer*, pages 59–67, 2002.

[30] DOORS. `http://www.telelogic.com/products/synergy/integrations/doors_integration.cfm`.

[31] Unified Modeling Language. `http://www.uml.org/`.

[32] System Modeling Language. `http://www.sysml.org/`.

[33] Ameos. `http://www.aonix.com`.

[34] ETAS. `http://en.etasgroup.com/products/ascet/`.

[35] Reactis. `http://www.reactive-systems.com/`.

[36] Safety Test Builder. `http://www.tni-valiosys.com/`.

[37] METROPOLIS. `http://www.gigascale.org/metropolis/`.

[38] Simple Scalar. `http://www.simplescalar.com/`.

[39] Verfication Group at Stanford. `http://verify.stanford.edu/`.

[40] MOCHA. `http://www.cis.upenn.edu/ mocha/`.

[41] Model Checking with SPIN. `http://spinroot.com/spin/whatispin.html`.

[42] Esterel Technologies. `http://www.esterel-technologies.com/`.

[43] B. Potter. Achieving Six Sigma Quality Through the use of Automatic Code Generation. *Mathworks International Aerospace and Defense Conference, Natick, MA*, June 2005.

[44] T. Katayama and A. Ohata Y. Uematsu. Production Code Generation for Engine Control System. *Mathworks International Automotive Conference, Natick, MA*, June 2004.

[45] R. Humphrey. Model Based Development for Automotive Body Systems. *Mathworks International Automotive Conference, Natick, MA*, June 2005.

[46] A. Ferrari. Platform and Model Based Design methodologies for Embedded Systems. *Workshop on Embedded Systems, United Technologies Research Center, East Hartford, CT*, May 2005.